

КАФЕДРА Системы обработки информации и управления

Тема: «Предобработка и классификация текста»

1

1) Введение

Обработка естественного языка является одним из важных направлений машинного обучения, которое используется для анализа, преобразования и классификации текстовой информации. Перед применением моделей машинного обучения текстовые данные необходимо представить в числовом виде, так как большинство алгоритмов классификации работают с числовыми признаками. Для этого используются методы предобработки текста, а также различные способы векторизации, позволяющие преобразовать слова и документы в векторные представления.

Цель данной лабораторной работы — изучение методов предобработки и классификации текстов. В ходе работы были рассмотрены основные этапы лингвистической обработки текста: токенизация, частеречная разметка, лемматизация, выделение именованных сущностей и синтаксический разбор предложения. Для выполнения этих этапов использовалась библиотека spaCy.

Во второй части лабораторной работы была решена задача классификации текстов на основе набора данных 20 Newsgroups. Классификация выполнялась двумя способами: с использованием векторизации TF-IDF и с использованием модели Word2Vec. После обучения моделей была выполнена оценка качества классификации и проведено сравнение полученных результатов.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо выполнить две основные части. В первой части требуется для произвольного предложения или текста выполнить основные этапы предобработки: токенизацию, частеречную разметку, лемматизацию, выделение именованных сущностей и синтаксический разбор предложения.

Во второй части необходимо выбрать произвольный набор данных, предназначенный для классификации текстов, и решить задачу классификации двумя способами. Первый способ основан на использовании CountVectorizer или TfidfVectorizer. Второй способ

основан на использовании моделей векторного представления слов, таких как Word2Vec, GloVe или fastText. После обучения моделей необходимо сравнить качество полученных результатов.

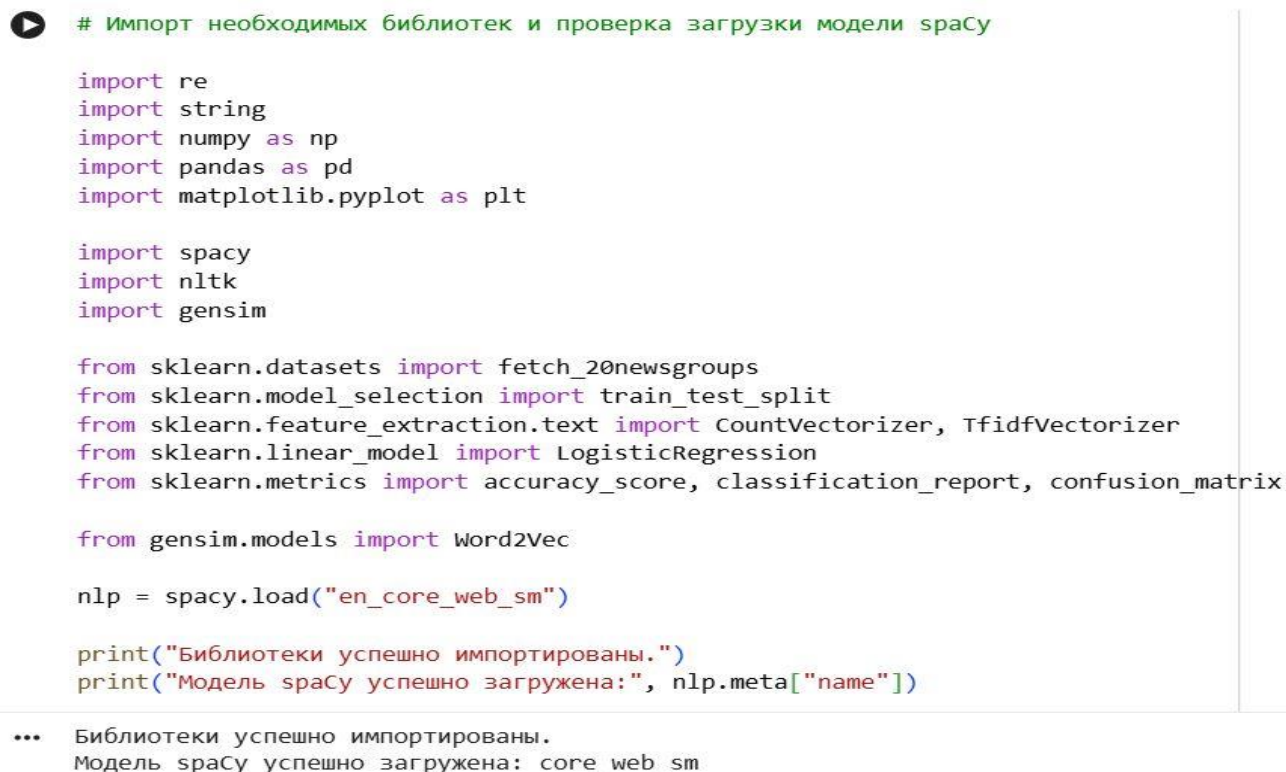
В данной работе для первой части был выбран небольшой англоязычный текст о компании Apple и системе искусственного интеллекта. Для второй части использовался набор данных 20 Newsgroups, из которого были выбраны четыре класса: comp.graphics, rec.sport.baseball, sci.space и talk.politics.misc. Для классификации использовались модели TF-IDF + LogisticRegression и Word2Vec + LogisticRegression.

3) Текст программы и экранные формы с примерами выполнения

В данном разделе приведены основные этапы программной реализации лабораторной работы. Все вычисления были выполнены в среде Google Colab с использованием языка программирования Python.

Для выполнения предобработки текста использовалась библиотека spaCy, которая позволяет выполнять токенизацию, частеречную разметку, лемматизацию, выделение именованных сущностей и синтаксический разбор предложения. Для работы с табличными данными использовалась библиотека pandas, для численных вычислений — numpy, для построения графиков — matplotlib.

Для решения задачи классификации текстов использовались средства библиотеки scikit-learn: набор данных 20 Newsgroups, TfidfVectorizer, модель LogisticRegression и метрики качества classification_report, accuracy_score и confusion_matrix. Для построения модели Word2Vec использовалась библиотека gensim.



```
# Импорт необходимых библиотек и проверка загрузки модели spaCy

import re
import string
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import spacy
import nltk
import gensim

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from gensim.models import Word2Vec

nlp = spacy.load("en_core_web_sm")

print("Библиотеки успешно импортированы.")
print("Модель spaCy успешно загружена:", nlp.meta["name"])

... Библиотеки успешно импортированы.
Модель spaCy успешно загружена: core_web_sm
```

Рисунок 1 — Импорт необходимых библиотек и проверка модели spaCy

3.1. Задание исходного текста для предобработки

На первом этапе был задан исходный текст, для которого далее выполнялись основные этапы предобработки. Текст состоит из двух предложений на английском языке и содержит несколько объектов, которые могут быть распознаны как именованные сущности, например Apple, California и 2024.

После задания текста он был обработан с помощью модели spaCy. В результате были выведены сам исходный текст, количество символов, количество токенов и количество предложений. Полученный результат показывает, что текст содержит 170 символов, 31 токен и 2 предложения. Данная информация подтверждает, что текст успешно загружен и готов к дальнейшей лингвистической обработке.

```
# Задание исходного текста для выполнения этапов предобработки

text = """
Apple announced a new artificial intelligence system in California in 2024.
The company plans to integrate this technology into its products for users around the world.
"""

doc = nlp(text)

print("Исходный текст:")
print(text)

print("\nКоличество символов:", len(text))
print("Количество токенов:", len(doc))
print("Количество предложений:", len(list(doc.sents)))

*** Исходный текст:

Apple announced a new artificial intelligence system in California in 2024.
The company plans to integrate this technology into its products for users around the world.

Количество символов: 170
Количество токенов: 31
Количество предложений: 2
```

Рисунок 2 — Исходный текст для предобработки

3.2. Токенизация текста

После задания исходного текста была выполнена токенизация. Токенизация представляет собой процесс разбиения текста на отдельные элементы — токены. В качестве токенов могут выступать слова, знаки препинания и служебные элементы текста.

В ходе выполнения данного этапа для каждого токена были выведены его текстовое значение, индекс в документе, признак пунктуации и признак пробела. Полученный результат показывает, что spaCy корректно разделил исходный текст на отдельные токены. При этом знаки препинания также были выделены как отдельные токены, что является стандартным поведением при лингвистической обработке текста.

```
# Выполнение токенизации исходного текста

tokens_data = []

for token in doc:
    tokens_data.append({
        "Токен": token.text,
        "Индекс": token.i,
        "Является пунктуацией": token.is_punct,
        "Является пробелом": token.is_space
    })

tokens_df = pd.DataFrame(tokens_data)

print("Результат токенизации текста:")
display(tokens_df)
```

Рисунок 3 — Код выполнения токенизации текста

Результат токенизации текста:

	Токен	Индекс	Является пунктуацией	Является пробелом
0	\n	0	False	True
1	Apple	1	False	False
2	announced	2	False	False
3	a	3	False	False
4	new	4	False	False
5	artificial	5	False	False
6	intelligence	6	False	False
7	system	7	False	False
8	in	8	False	False
9	California	9	False	False
10	in	10	False	False
11	2024	11	False	False
12	.	12	True	False
13	\n	13	False	True
14	The	14	False	False
15	company	15	False	False
16	plans	16	False	False
17	to	17	False	False
18	integrate	18	False	False
19	this	19	False	False
20	technology	20	False	False
21	into	21	False	False
22	its	22	False	False
23	products	23	False	False
24	for	24	False	False
25	users	25	False	False
26	around	26	False	False
27	the	27	False	False
28	world	28	False	False
29	.	29	True	False
30	\n	30	False	True

Рисунок 4 — Результат токенизации текста

3.3. Частеречная разметка и лемматизация

На следующем этапе была выполнена частеречная разметка и лемматизация токенов. Частеречная разметка позволяет определить грамматическую категорию каждого токена, например существительное, глагол, прилагательное, предлог или знак пунктуации. Лемматизация используется для приведения слова к его начальной форме.

В результате работы программы для каждого токена были выведены его исходная форма, лемма, часть речи, подробный грамматический тег и описание этого тега. Полученный результат показывает, что библиотека spaCy корректно определила части речи для слов исходного текста. Например, слово `announced` было приведено к лемме `announce` и определено как глагол, а слово `products` было приведено к лемме `product` и определено как существительное во множественном числе.

```
▶ # Выполнение частеречной разметки и лемматизации токенов

pos_lemma_data = []

for token in doc:
    if not token.is_space:
        pos_lemma_data.append({
            "Токен": token.text,
            "Лемма": token.lemma_,
            "Часть речи": token.pos_,
            "Подробный тег": token.tag_,
            "Описание тега": spacy.explain(token.tag_)
        })

pos_lemma_df = pd.DataFrame(pos_lemma_data)

print("Результат частеречной разметки и лемматизации:")
display(pos_lemma_df)
```

Рисунок 5 — Код частеречной разметки и лемматизации

Результат частеречной разметки и лемматизации:

	Токен	Лемма	Часть речи	Подробный тег	Описание тега
0	Apple	Apple	PROPN	NNP	noun, proper singular
1	announced	announce	VERB	VBD	verb, past tense
2	a	a	DET	DT	determiner
3	new	new	ADJ	JJ	adjective (English), other noun-modifier (Chin...
4	artificial	artificial	ADJ	JJ	adjective (English), other noun-modifier (Chin...
5	intelligence	intelligence	NOUN	NN	noun, singular or mass
6	system	system	NOUN	NN	noun, singular or mass
7	in	in	ADP	IN	conjunction, subordinating or preposition
8	California	California	PROPN	NNP	noun, proper singular
9	in	in	ADP	IN	conjunction, subordinating or preposition
10	2024	2024	NUM	CD	cardinal number
11	.	.	PUNCT	.	punctuation mark, sentence closer
12	The	the	DET	DT	determiner
13	company	company	NOUN	NN	noun, singular or mass
14	plans	plan	VERB	VBZ	verb, 3rd person singular present
15	to	to	PART	TO	infinitival "to"
16	integrate	integrate	VERB	VB	verb, base form
17	this	this	DET	DT	determiner
18	technology	technology	NOUN	NN	noun, singular or mass
19	into	into	ADP	IN	conjunction, subordinating or preposition
20	its	its	PRON	PRP\$	pronoun, possessive
21	products	product	NOUN	NNS	noun, plural
22	for	for	ADP	IN	conjunction, subordinating or preposition
23	users	user	NOUN	NNS	noun, plural
24	around	around	ADP	IN	conjunction, subordinating or preposition
25	the	the	DET	DT	determiner
26	world	world	NOUN	NN	noun, singular or mass
27	.	.	PUNCT	.	punctuation mark, sentence closer

Рисунок 6 — Результат частеречной разметки и лемматизации

3.4. Выделение именованных сущностей

После частеречной разметки было выполнено выделение именованных сущностей. Данный этап позволяет находить в тексте объекты определенных типов, например названия организаций, географические объекты, даты, имена людей и другие значимые сущности.

В результате работы программы были найдены три именованные сущности: Apple, California и 2024. Сущность Apple была определена как организация, California — как геополитический объект, а 2024 — как дата. Для каждой сущности также были выведены ее тип, описание типа, начальная и конечная позиция в исходном тексте. Полученный результат показывает, что модель spaCy корректно распознала основные именованные сущности в выбранном тексте.

```
# Выделение именованных сущностей в исходном тексте

entities_data = []

for ent in doc.ents:
    entities_data.append({
        "Сущность": ent.text,
        "Тип": ent.label_,
        "Описание типа": spacy.explain(ent.label_),
        "Начальная позиция": ent.start_char,
        "Конечная позиция": ent.end_char
    })

entities_df = pd.DataFrame(entities_data)

print("Распознанные именованные сущности:")
display(entities_df)
```

Рисунок 7 — Код выделения именованных сущностей

Распознанные именованные сущности:

	Сущность	Тип	Описание типа	Начальная позиция	Конечная позиция
0	Apple	ORG	Companies, agencies, institutions, etc.	1	6
1	California	GPE	Countries, cities, states	57	67
2	2024	DATE	Absolute or relative dates or periods	71	75

Рисунок 8 — Результат выделения именованных сущностей

3.5. Синтаксический разбор предложения

На заключительном этапе первой части был выполнен синтаксический разбор предложений исходного текста. Синтаксический разбор позволяет определить связи между словами в предложении и выделить главное слово для каждого токена. Также для каждого токена определяется его синтаксическая роль, например подлежащее, сказуемое, дополнение, модификатор или знак пунктуации.

В результате работы программы для каждого токена были выведены его текстовое значение, лемма, часть речи, синтаксическая роль, описание этой роли и главное слово, от которого зависит данный токен. Полученный результат показывает структуру предложений исходного текста и позволяет увидеть грамматические зависимости между словами. Например, для глаголов `announced` и `plans` были определены зависимые слова, что отражает синтаксическую организацию предложений.

```
▶ # Выполнение синтаксического разбора предложений исходного текста

parse_data = []

for token in doc:
    if not token.is_space:
        parse_data.append({
            "Токен": token.text,
            "Лемма": token.lemma_,
            "Часть речи": token.pos_,
            "Синтаксическая роль": token.dep_,
            "Описание роли": spacy.explain(token.dep_),
            "Главное слово": token.head.text
        })

parse_df = pd.DataFrame(parse_data)

print("Результат синтаксического разбора предложений:")
display(parse_df)
```

Рисунок 9 — Код синтаксического разбора предложений

Результат синтаксического разбора предложений:

	Токен	Лемма	Часть речи	Синтаксическая роль	Описание роли	Главное слово
0	Apple	Apple	PROPN	nsubj	nominal subject	announced
1	announced	announce	VERB	ROOT	root	announced
2	a	a	DET	det	determiner	system
3	new	new	ADJ	amod	adjectival modifier	system
4	artificial	artificial	ADJ	amod	adjectival modifier	system
5	intelligence	intelligence	NOUN	compound	compound	system
6	system	system	NOUN	dobj	direct object	announced
7	in	in	ADP	prep	prepositional modifier	system
8	California	California	PROPN	pobj	object of preposition	in
9	in	in	ADP	prep	prepositional modifier	announced
10	2024	2024	NUM	pobj	object of preposition	in
11	.	.	PUNCT	punct	punctuation	announced
12	The	the	DET	det	determiner	company
13	company	company	NOUN	nsubj	nominal subject	plans
14	plans	plan	VERB	ROOT	root	plans
15	to	to	PART	aux	auxiliary	integrate
16	integrate	integrate	VERB	xcomp	open clausal complement	plans
17	this	this	DET	det	determiner	technology
18	technology	technology	NOUN	dobj	direct object	integrate
19	into	into	ADP	prep	prepositional modifier	integrate
20	its	its	PRON	poss	possession modifier	products
21	products	product	NOUN	pobj	object of preposition	into
22	for	for	ADP	prep	prepositional modifier	products
23	users	user	NOUN	pobj	object of preposition	for
24	around	around	ADP	prep	prepositional modifier	users
25	the	the	DET	det	determiner	world
26	world	world	NOUN	pobj	object of preposition	around
27	.	.	PUNCT	punct	punctuation	plans

Рисунок 10 — Результат синтаксического разбора предложений

3.6. Загрузка и анализ набора данных

После выполнения предобработки отдельного текста была начата вторая часть лабораторной работы, связанная с классификацией текстовых данных. Для решения задачи был выбран набор данных 20 Newsgroups, который содержит текстовые документы, относящиеся к различным тематическим категориям.

В данной работе использовались четыре класса: `comp.graphics`, `rec.sport.baseball`, `sci.space` и `talk.politics.misc`. После загрузки набора данных были выведены количество текстов, количество классов, названия классов, распределение документов по классам, а также пример одного текстового документа. Всего было загружено 3729 текстовых сообщений, относящихся к четырем тематическим категориям.

```
# Загрузка набора данных 20 Newsgroups для задачи классификации текстов

categories = [
    "rec.sport.baseball",
    "sci.space",
    "comp.graphics",
    "talk.politics.misc"
]

dataset = fetch_20newsgroups(
    subset="all",
    categories=categories,
    remove=("headers", "footers", "quotes"),
    shuffle=True,
    random_state=42
)

texts = dataset.data
labels = dataset.target
target_names = dataset.target_names

print("Набор данных успешно загружен.")
print("Количество текстов:", len(texts))
print("Количество классов:", len(target_names))
print("Названия классов:", target_names)

class_distribution = pd.Series(labels).value_counts().sort_index()
class_distribution_df = pd.DataFrame({
    "Класс": target_names,
    "Количество документов": class_distribution.values
})

print("\nРаспределение документов по классам:")
display(class_distribution_df)

print("\nПример текста:")
print(texts[0][:1000])
print("\nКласс примера:", target_names[labels[0]])
```

Рисунок 11 — Код загрузки набора данных 20 Newsgroups

```
... Набор данных успешно загружен.  
Количество текстов: 3729  
Количество классов: 4  
Названия классов: ['comp.graphics', 'rec.sport.baseball', 'sci.space', 'talk.politics.misc']
```

Распределение документов по классам:

	Класс	Количество документов	
0	comp.graphics	973	
1	rec.sport.baseball	994	
2	sci.space	987	
3	talk.politics.misc	775	

Пример текста:

```
Are you kidding? I'm stuck with the Toronto SkyDome, where their idea  
of a 7th inning stretch is that "Blue Jays" song where everyone gets  
to yell: "Okay, okay, Blue Jays, Blue Jays, Let's Play Ball!"
```

```
Wow.. what genius did it take to compose that one, to outshine the  
old classic. And there are women on the field to "lead the crowd".
```

Класс примера: `rec.sport.baseball`

Рисунок 12 — Результат загрузки и анализа набора данных

3.7. Разделение данных на обучающую и тестовую выборки

После загрузки набора данных было выполнено разделение текстов на обучающую и тестовую выборки. Такое разделение необходимо для того, чтобы обучить модели на одной части данных, а затем проверить качество классификации на другой части данных, которая не использовалась при обучении.

В данной работе 75% данных были использованы для обучения моделей, а 25% — для тестирования. Для сохранения одинакового распределения классов в обучающей и тестовой выборках использовался параметр `stratify`. В результате размер обучающей выборки составил 2796 документов, а размер тестовой выборки — 933 документа. Распределение документов по классам после разделения осталось сбалансированным.

▶ # Разделение набора данных на обучающую и тестовую выборки

```
x_train, x_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

print("Разделение данных выполнено.")
print("Размер обучающей выборки:", len(x_train))
print("Размер тестовой выборки:", len(x_test))

train_distribution = pd.Series(y_train).value_counts().sort_index()
test_distribution = pd.Series(y_test).value_counts().sort_index()

split_distribution_df = pd.DataFrame({
    "Класс": target_names,
    "Обучающая выборка": train_distribution.values,
    "Тестовая выборка": test_distribution.values
})

print("\nРаспределение документов после разделения:")
display(split_distribution_df)
```

Рисунок 13 — Код разделения данных на обучающую и тестовую выборки

Разделение данных выполнено.
Размер обучающей выборки: 2796
Размер тестовой выборки: 933

Распределение документов после разделения:

	Класс	Обучающая выборка	Тестовая выборка
0	comp.graphics	730	243
1	rec.sport.baseball	745	249
2	sci.space	740	247
3	talk.politics.misc	581	194

Рисунок 14 — Результат разделения данных на обучающую и тестовую выборки

3.8. Классификация текста на основе TF-IDF

Первый способ классификации был основан на использовании метода TF-IDF. Данный метод преобразует текстовые документы в числовые векторы, учитывая не только частоту появления слова в документе, но и его значимость относительно всего корпуса текстов. В данной работе использовался векторизатор `TfidfVectorizer` с удалением английских стоп-слов, ограничением количества признаков и учетом униграмм и биграмм.

После преобразования текстов в TF-IDF-векторы была обучена модель `LogisticRegression`. Размер матрицы признаков обучающей выборки составил 2796×10000 , а размер матрицы признаков тестовой выборки — 933×10000 . По результатам тестирования точность модели составила 0.8928. Отчет о классификации показывает значения `precision`, `recall` и `F1-score` для каждого из четырех классов.

```
# Обучение модели классификации на основе TF-IDF и LogisticRegression

tfidf_vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    max_features=10000,
    ngram_range=(1, 2)
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

tfidf_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

tfidf_model.fit(X_train_tfidf, y_train)
y_pred_tfidf = tfidf_model.predict(X_test_tfidf)

tfidf_accuracy = accuracy_score(y_test, y_pred_tfidf)

print("Модель TF-IDF + LogisticRegression обучена.")
print("Размер матрицы признаков обучающей выборки:", X_train_tfidf.shape)
print("Размер матрицы признаков тестовой выборки:", X_test_tfidf.shape)
print("Accuracy модели TF-IDF:", round(tfidf_accuracy, 4))

print("\nОтчет о классификации:")
print(classification_report(y_test, y_pred_tfidf, target_names=target_names))
```

Рисунок 15 — Код обучения модели на основе TF-IDF

```

... Модель TF-IDF + LogisticRegression обучена.
Размер матрицы признаков обучающей выборки: (2796, 10000)
Размер матрицы признаков тестовой выборки: (933, 10000)
Accuracy модели TF-IDF: 0.8928

```

Отчет о классификации:

	precision	recall	f1-score	support
comp.graphics	0.94	0.93	0.94	243
rec.sport.baseball	0.91	0.85	0.88	249
sci.space	0.81	0.94	0.87	247
talk.politics.misc	0.94	0.84	0.89	194
accuracy			0.89	933
macro avg	0.90	0.89	0.89	933
weighted avg	0.90	0.89	0.89	933

Рисунок 16 — Результат обучения и оценки модели на основе TF-IDF

3.9. Матрица ошибок для модели TF-IDF

Для более подробного анализа качества первой модели была построена матрица ошибок. Матрица ошибок показывает, сколько документов каждого истинного класса были отнесены моделью к каждому из возможных классов. Значения на главной диагонали соответствуют правильно классифицированным документам, а остальные значения показывают ошибки классификации.

По матрице ошибок видно, что модель TF-IDF + LogisticRegression в большинстве случаев правильно определяет классы документов. Наибольшее количество правильных предсказаний находится на главной диагонали матрицы. Ошибки в основном возникают между тематически близкими или менее однозначными текстами, что является ожидаемым результатом для задачи классификации текстов.

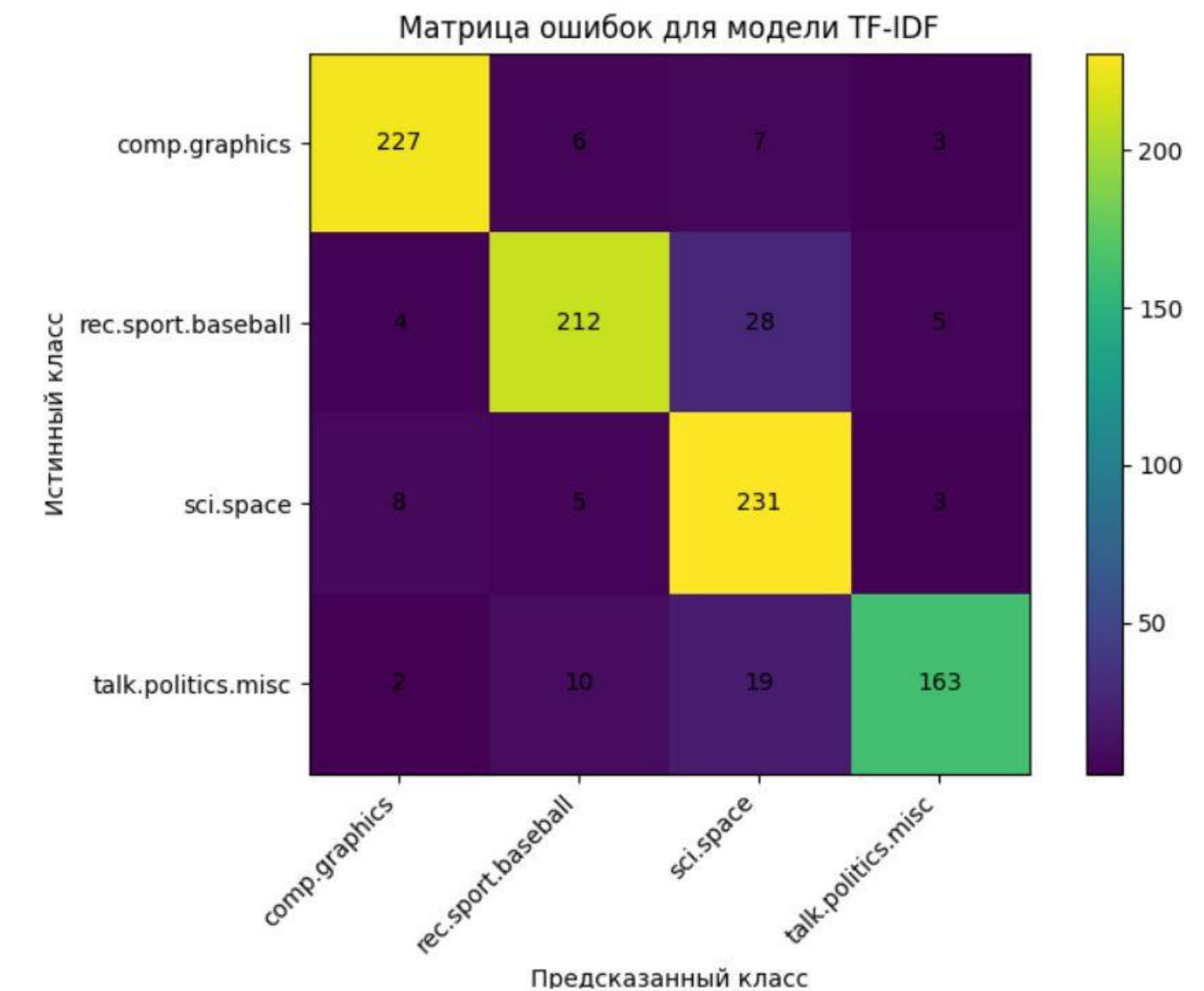


Рисунок 17 — Матрица ошибок для модели TF-IDF

3.10. Предварительная обработка текстов для Word2Vec

Второй способ классификации был основан на использовании модели Word2Vec. Перед обучением данной модели тексты были предварительно обработаны: все символы были приведены к нижнему регистру, из текста были удалены знаки препинания, цифры и специальные символы, после чего текст был разделен на отдельные токены.

Также из полученных токенов были удалены слишком короткие слова длиной менее трех символов. Такая обработка позволяет уменьшить количество шумовых элементов и подготовить корпус текстов для обучения модели Word2Vec. В результате были

сформированы токенизированные представления обучающей и тестовой выборок, которые далее использовались для построения векторных представлений документов.

```
▶ # Предварительная обработка текстов для обучения модели Word2Vec

def preprocess_for_word2vec(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = text.split()
    tokens = [token for token in tokens if len(token) > 2]
    return tokens

X_train_tokens = [preprocess_for_word2vec(text) for text in X_train]
X_test_tokens = [preprocess_for_word2vec(text) for text in X_test]

print("Предварительная обработка текстов для Word2Vec выполнена.")
print("Количество документов в обучающей выборке:", len(X_train_tokens))
print("Количество документов в тестовой выборке:", len(X_test_tokens))

print("\nПример исходного текста:")
print(X_train[0][:500])

print("\nПример токенизированного текста:")
print(X_train_tokens[0][:50])
```

Рисунок 18 — Код предварительной обработки текстов для Word2Vec

```
... Предварительная обработка текстов для Word2Vec выполнена.
Количество документов в обучающей выборке: 2796
Количество документов в тестовой выборке: 933

Пример исходного текста:
I have a routine that changes the color (RGB) attributes on my
VGA adapter, but it doesn't work in the mode that I need.
Specifically 68 hex. An obscure mode, of course, but I need to
change the zillions of colors to 64 shade greyscale, but I do
not have the correct memory address for the pointer I need.

PLEASE, someone, I need the starting address, or maybe somewhere
I can find it. Thank you.

Пример токенизированного текста:
['have', 'routine', 'that', 'changes', 'the', 'color', 'rgb', 'attributes', 'vga', 'adapter', 'but', 'doesn', 'work',
```

Рисунок 19 — Результат предварительной обработки текстов для Word2Vec

3.11. Обучение модели Word2Vec и формирование векторов документов

После предварительной обработки текстов была обучена модель Word2Vec на токенизированной обучающей выборке. Модель Word2Vec формирует векторные представления слов на основе их контекстной близости, то есть слова, которые встречаются в похожих контекстах, получают близкие векторы.

В данной работе использовалась модель Word2Vec с размерностью векторов 100, окном контекста 5 и алгоритмом Skip-gram. После обучения модели каждый документ был преобразован в один вектор путем усреднения векторов входящих в него слов. В результате были получены матрицы признаков размером 2796×100 для обучающей выборки и 933×100 для тестовой выборки. Размер словаря обученной модели Word2Vec составил 15597 слов.

```
▶ # Обучение модели Word2Vec и преобразование документов в векторные представления

word2vec_model = Word2Vec(
    sentences=X_train_tokens,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    sg=1,
    epochs=20,
    seed=42
)

def document_vector(tokens, model):
    vectors = []

    for token in tokens:
        if token in model.wv:
            vectors.append(model.wv[token])

    if len(vectors) == 0:
        return np.zeros(model.vector_size)

    return np.mean(vectors, axis=0)

X_train_w2v = np.array([document_vector(tokens, word2vec_model) for tokens in X_train_tokens])
X_test_w2v = np.array([document_vector(tokens, word2vec_model) for tokens in X_test_tokens])

print("Модель Word2Vec обучена.")
print("Размер словаря Word2Vec:", len(word2vec_model.wv))
print("Размерность векторов слов:", word2vec_model.vector_size)
print("Размер матрицы признаков обучающей выборки:", X_train_w2v.shape)
print("Размер матрицы признаков тестовой выборки:", X_test_w2v.shape)

print("\nНаиболее похожие слова для слова space:")
print(word2vec_model.wv.most_similar("space", topn=10))
```

Рисунок 20 — Код обучения модели Word2Vec и формирования векторов документов

```
*** Модель Word2Vec обучена.  
Размер словаря Word2Vec: 15597  
Размерность векторов слов: 100  
Размер матрицы признаков обучающей выборки: (2796, 100)  
Размер матрицы признаков тестовой выборки: (933, 100)  
  
Наиболее похожие слова для слова space:  
[('sci', 0.5749902129173279), ('station', 0.5595186948776245), ('exploration', 0.5426974892616272), ('dawn', 0.5396639704704285), ('nasda', 0.5303415656089783), ('missions',
```

Рисунок 21 — Результат обучения модели Word2Vec и формирования векторов документов

3.12. Классификация текста на основе Word2Vec

После формирования векторных представлений документов была обучена модель LogisticRegression для классификации текстов на основе признаков Word2Vec. В отличие от TF-IDF, в данном подходе каждый документ представлен плотным числовым вектором размерности 100, который был получен путем усреднения векторов слов, входящих в документ.

После обучения модель была протестирована на тестовой выборке. Точность классификации составила 0.8907. Отчет о классификации показывает, что модель Word2Vec + LogisticRegression также достаточно хорошо справляется с задачей разделения документов на четыре тематические категории. Полученный результат близок к качеству модели на основе TF-IDF.

▶ # Обучение модели классификации на основе векторных представлений Word2Vec

```
word2vec_classifier = LogisticRegression(  
    max_iter=1000,  
    random_state=42  
)  
  
word2vec_classifier.fit(X_train_w2v, y_train)  
y_pred_w2v = word2vec_classifier.predict(X_test_w2v)  
  
word2vec_accuracy = accuracy_score(y_test, y_pred_w2v)  
  
print("Модель Word2Vec + LogisticRegression обучена.")  
print("Accuracy модели Word2Vec:", round(word2vec_accuracy, 4))  
  
print("\nОтчет о классификации:")  
print(classification_report(y_test, y_pred_w2v, target_names=target_names))
```

Рисунок 22 — Код обучения модели классификации на основе Word2Vec

... Модель Word2Vec + LogisticRegression обучена.
Accuracy модели Word2Vec: 0.8907

Отчет о классификации:

	precision	recall	f1-score	support
comp.graphics	0.93	0.91	0.92	243
rec.sport.baseball	0.86	0.96	0.91	249
sci.space	0.89	0.82	0.85	247
talk.politics.misc	0.90	0.87	0.88	194
accuracy			0.89	933
macro avg	0.89	0.89	0.89	933
weighted avg	0.89	0.89	0.89	933

Рисунок 23 — Результат обучения и оценки модели на основе Word2Vec

3.13. Матрица ошибок для модели Word2Vec

Для анализа ошибок второй модели была построена матрица ошибок для классификатора Word2Vec + LogisticRegression. Как и в случае с моделью TF-IDF, матрица показывает распределение правильных и ошибочных предсказаний по каждому классу.

По полученной матрице видно, что модель Word2Vec также правильно классифицирует большую часть документов, так как наибольшие значения расположены на главной диагонали. Ошибки классификации присутствуют, однако их количество невелико. Это подтверждает, что усредненные векторные представления слов могут быть использованы для решения задачи классификации текстов.

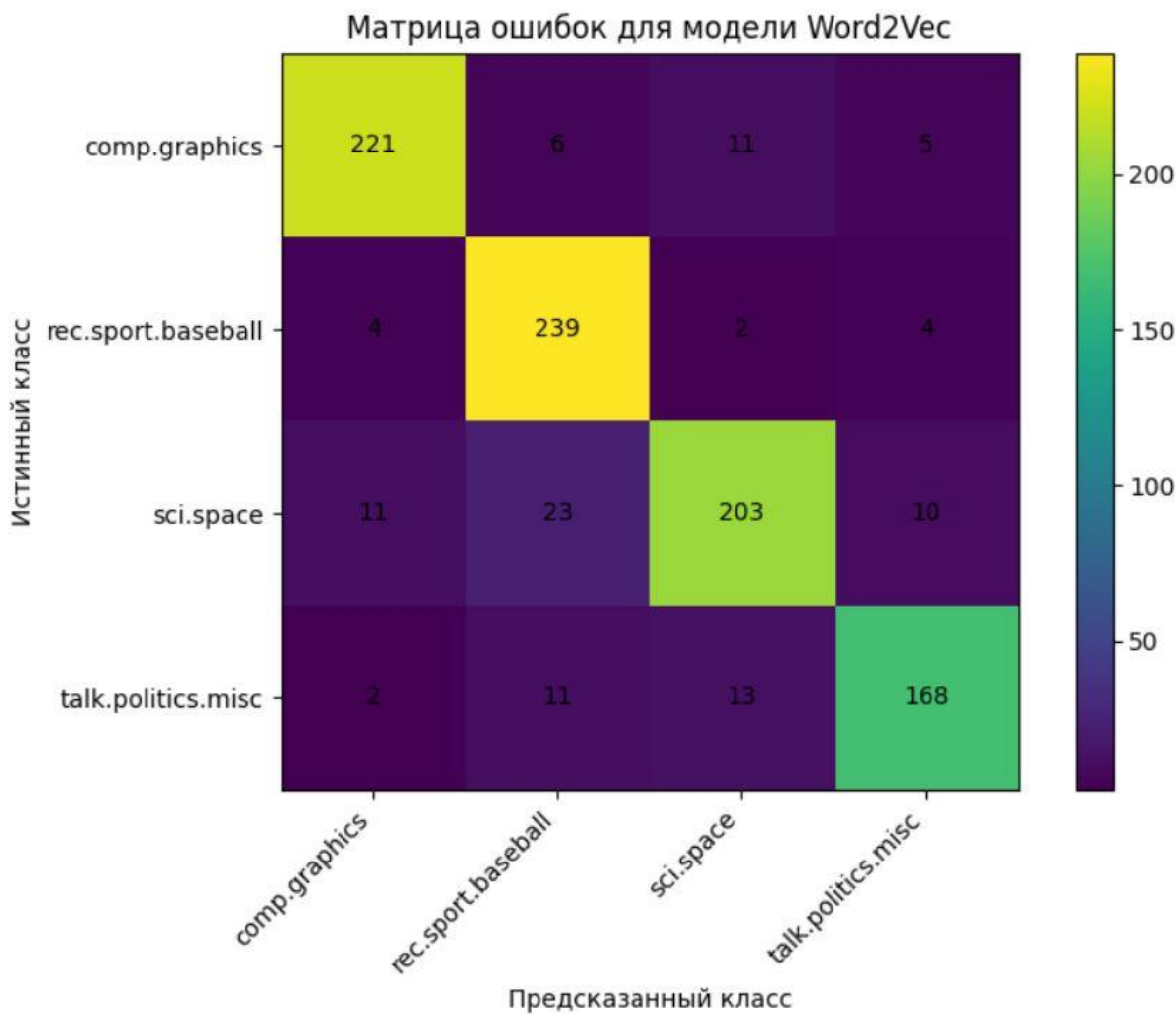


Рисунок 24 — Матрица ошибок для модели Word2Vec

3.14. Сравнение качества моделей

После обучения двух моделей было выполнено сравнение качества классификации. Для сравнения использовались основные метрики: Accuracy, Precision, Recall и F1-score. Эти метрики позволяют оценить качество модели не только по общей доле правильных ответов, но и с учетом качества классификации по отдельным классам.

Результаты сравнения показали, что обе модели имеют близкое качество. Модель TF-IDF + LogisticRegression получила Accuracy 0.8928, Precision 0.8975, Recall 0.8928 и F1-score 0.8932. Модель Word2Vec + LogisticRegression получила Accuracy 0.8907, Precision 0.8919, Recall 0.8907 и F1-score 0.8901. По значению Accuracy лучшей моделью оказалась TF-IDF + LogisticRegression.

```
# Сравнение качества моделей TF-IDF и Word2Vec

from sklearn.metrics import precision_recall_fscore_support

tfidf_precision, tfidf_recall, tfidf_f1, _ = precision_recall_fscore_support(
    y_test,
    y_pred_tfidf,
    average="weighted"
)

word2vec_precision, word2vec_recall, word2vec_f1, _ = precision_recall_fscore_support(
    y_test,
    y_pred_w2v,
    average="weighted"
)

comparison_df = pd.DataFrame({
    "Модель": [
        "TF-IDF + LogisticRegression",
        "Word2Vec + LogisticRegression"
    ],
    "Accuracy": [
        tfidf_accuracy,
        word2vec_accuracy
    ],
    "Precision": [
        tfidf_precision,
        word2vec_precision
    ],
    "Recall": [
        tfidf_recall,
        word2vec_recall
    ],
    "F1-score": [
        tfidf_f1,
        word2vec_f1
    ]
})

comparison_df[["Accuracy", "Precision", "Recall", "F1-score"]] = comparison_df[
    ["Accuracy", "Precision", "Recall", "F1-score"]
].round(4)

print("Сравнение качества моделей:")
display(comparison_df)

best_model = comparison_df.loc[comparison_df["Accuracy"].idxmax(), "Модель"]
print("\nЛучшая модель по Accuracy:", best_model)
```

Рисунок 25 — Код сравнения качества моделей

Сравнение качества моделей:

	Модель	Accuracy	Precision	Recall	F1-score
0	TF-IDF + LogisticRegression	0.8928	0.8975	0.8928	0.8932
1	Word2Vec + LogisticRegression	0.8907	0.8919	0.8907	0.8901

Лучшая модель по Accuracy: TF-IDF + LogisticRegression

Рисунок 26 — Результат сравнения качества моделей

3.15. Диаграмма сравнения Accuracy моделей

Для наглядного представления результатов была построена диаграмма сравнения Accuracy двух моделей. По оси абсцисс отображаются модели, а по оси ординат — значение Accuracy на тестовой выборке.

Из диаграммы видно, что обе модели показали почти одинаковое качество классификации. Модель TF-IDF + LogisticRegression имеет немного более высокое значение Accuracy, равное 0.8928, тогда как модель Word2Vec + LogisticRegression получила Accuracy 0.8907. Разница между моделями является небольшой, однако по итоговой метрике Accuracy первый способ оказался немного лучше.

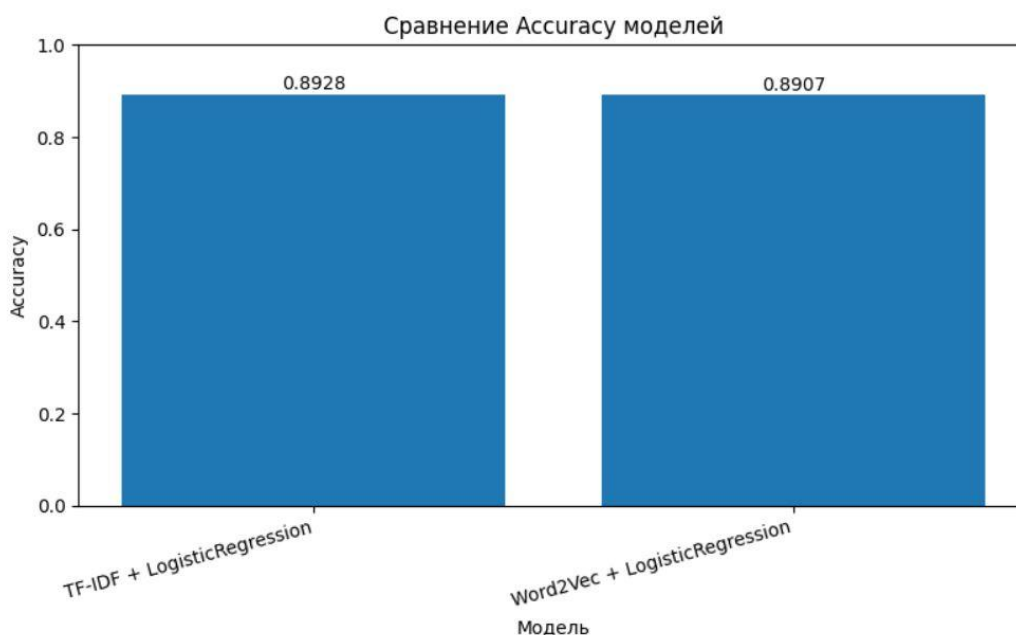


Рисунок 27 — Диаграмма сравнения Accuracy моделей

3.16. Формирование итоговой таблицы результатов

На заключительном этапе выполнения программы была сформирована итоговая таблица результатов лабораторной работы. В таблицу были включены основные характеристики выполненного эксперимента: название набора данных, количество классов, количество документов, размер обучающей и тестовой выборок, использованные методы классификации и значения Accurasy для каждой модели.

Итоговая таблица показывает, что в работе использовался набор данных 20 Newsgroups, содержащий 3729 документов и 4 класса. Для обучения использовались 2796 документов, а для тестирования — 933 документа. Первый метод TF-IDF + LogisticRegression показал Accuracy 0.8928, второй метод Word2Vec + LogisticRegression показал Accuracy 0.8907. По итогам сравнения лучшей моделью была признана TF-IDF + LogisticRegression.

Итоговая таблица результатов лабораторной работы:

	Показатель	Значение
0	Набор данных	20 Newsgroups
1	Количество классов	4
2	Количество документов	3729
3	Размер обучающей выборки	2796
4	Размер тестовой выборки	933
5	Метод 1	TF-IDF + LogisticRegression
6	Accuracy метода 1	0.8928
7	Метод 2	Word2Vec + LogisticRegression
8	Accuracy метода 2	0.8907
9	Лучшая модель	TF-IDF + LogisticRegression

Рисунок 28 — Итоговая таблица результатов лабораторной работы

4) Заключение

В ходе лабораторной работы были изучены и реализованы основные методы предобработки и классификации текстов. В первой части работы для выбранного англоязычного текста были выполнены токенизация, частеречная разметка, лемматизация, выделение именованных сущностей и синтаксический разбор предложений. Для выполнения этих этапов использовалась библиотека spaCy, которая позволила получить подробную лингвистическую информацию о каждом токене исходного текста.

Во второй части лабораторной работы была решена задача классификации текстов на основе набора данных 20 Newsgroups. Для эксперимента были выбраны четыре тематических класса: comp.graphics, rec.sport.baseball, sci.space и talk.politics.misc. Данные были разделены на обучающую и тестовую выборки, после чего были реализованы два подхода к классификации.

Первый подход был основан на использовании TfidfVectorizer и модели LogisticRegression. Второй подход использовал модель Word2Vec для формирования векторных представлений документов и классификатор LogisticRegression. В результате модель TF-IDF + LogisticRegression получила Accuracy 0.8928, а модель Word2Vec + LogisticRegression получила Accuracy 0.8907.

Сравнение результатов показало, что оба подхода позволяют достаточно эффективно решать задачу классификации текстов. При этом модель на основе TF-IDF показала немного более высокое значение Accuracy, поэтому по итогам эксперимента она была признана лучшей моделью. Таким образом, все поставленные задачи лабораторной работы были выполнены.